

GUIA SÊNIOR • CYBERSECURITY

Auditoria de Segurança em 9 Camadas

CHECKLIST SÊNIOR • 240+ VERIFICAÇÕES

Da skill automatizada ao pentest profissional.
O mapa completo que sua equipe precisa ter.

Secrets

Dependências

Código

Web

Auth

DB

Cloud

CI/CD

LGPD



Danillo Dantas
@danillodantas.ia

Conteúdo do guia

37 páginas. Use as cadências do Capítulo 11 para aplicar no seu ritmo.

00	Como usar este checklist	p. 03
01	Camada 1 — Secrets & credenciais	p. 04
02	Camada 2 — Dependências & supply chain	p. 06
03	Camada 3 — Código & lógica de aplicação	p. 08
04	Camada 4 — Web security & headers	p. 10
05	Camada 5 — Auth & AuthZ	p. 12
06	Camada 6 — Banco de dados & PII	p. 14
07	Camada 7 — Cloud & infraestrutura	p. 16
08	Camada 8 — CI/CD & supply chain interna	p. 18
09	Camada 9 — Compliance & LGPD	p. 20
10	A skill claude /security-audit	p. 22
11	Cadência de auditoria	p. 25
12	Próximos passos	p. 27
A	Apêndice — Glossário	p. 29
B	Apêndice — SAST · DAST · IAST · RASP	p. 31
C	Apêndice — Templates práticos	p. 33
D	Apêndice — 8 direitos LGPD na prática	p. 35

— COMO USAR

Como ler este checklist

Este documento é uma **referência operacional**. Não foi feito pra ser lido de capa a capa numa sentada — foi feito pra ser consultado quando você for atacar uma camada específica.

NÍVEIS DE CADA ITEM

Cada check tem um nível indicando o esforço/maturidade necessária:

- BÁSICO** Mínimo aceitável. Se falta, é red flag imediato. Implementa em horas.
- MÉDIO** Padrão de mercado. Se você cobra do cliente, precisa ter. Implementa em dias.
- SÊNIOR** Maturidade avançada. Diferencia equipe sênior de equipe júnior. Implementa em semanas.

O QUE ESTE CHECKLIST NÃO SUBSTITUI

Seja honesto sobre os limites:

- **Pentest profissional** — humano criativo acha vetores que checklist nenhum prevê
- **Threat modeling** — é discussão de arquitetura, não verificação
- **Consultoria sênior** — interpretação de risco no contexto do seu negócio
- **SIEM & runtime detection** — você precisa detectar o atacante já dentro

GLOSSÁRIO RÁPIDO

BOLA · Broken Object Level Authorization · **BFLA** · Broken Function Level Authorization · **SSRF** · Server-Side Request Forgery · **RLS** · Row Level Security · **SBOM** · Software Bill of Materials · **SLSA** · Supply-chain Levels for Software Artifacts · **CSP** · Content Security Policy · **HSTS** · HTTP Strict Transport Security · **DPA** · Data Processing Agreement · **DPIA** · Data Protection Impact Assessment.

Glossário completo no [Apêndice A](#).

SECRETS & CREDENCIAIS

Não é só API key **hardcoded**

O QUE É

Secret é qualquer dado que prova que o portador tem direito a acessar algo: API keys, tokens JWT, senhas de banco, certificados, chaves SSH, credenciais SMTP. Vazamento de secret é vetor número um em data breaches públicos — IBM 2024 reporta que credenciais comprometidas foram a porta de entrada em **15% de todos os breaches**, com tempo médio de detecção de 292 dias.

POR QUE IMPORTA PRO NEGÓCIO

Uma única AWS key vazada num commit público de 2019 pode estar sendo usada hoje pra minerar crypto na sua conta — você só descobre quando chega a fatura de US\$ 80 mil. JWT secret previsível em produção permite atacante forjar qualquer sessão. Chave SMTP roubada vira spam em massa enviado pelo seu domínio, queimando reputação de e-mail por meses.

Checklist · 28 verificações

- ☐ **B** .env está no .gitignore e nunca foi commitado em nenhum branch.
- ☐ **B** Histórico completo do Git foi escaneado com gitleaks ou trufflehog, não só HEAD.
- ☐ **B** Pre-commit hook (husky + gitleaks) bloqueia commit que contenha secret.
- ☐ **B** Secrets diferentes entre dev, staging e prod. Sem reaproveitamento.
- ☐ **B** JWT signing secret tem 256 bits ou mais e é gerado com CSPRNG.
- ☐ **B** Senhas de banco e API keys estão em variáveis de ambiente do runtime, não em arquivos versionados.
- ☐ **M** Secrets em produção vivem num secret manager (AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, Doppler).
- ☐ **M** AWS keys têm rotação automatizada via Secrets Manager rotation Lambda.
- ☐ **M** Logs estruturados removem campos sensíveis automaticamente (Pino redact, Winston format).
- ☐ **M** Sentry/Datadog tem PII scrubbing configurado e testado com payload conhecido.
- ☐ **M** URLs em logs nunca contêm tokens em query string (todos via header Authorization).
- ☐ **M** Headers Authorization, Cookie, X-API-Key estão na lista de redação dos logs.
- ☐ **M** CI/CD nunca printa env em job logs (echo "\$AWS_SECRET" proibido por hook).
- ☐ **M** Banco e Redis usam senhas únicas; sem credencial padrão (postgres/postgres).
- ☐ **M** Existe inventário escrito de todas as credenciais em uso, com responsável e data da última rotação.
- ☐ **M** SSH keys de produção são por pessoa, jamais compartilhadas; ~/.ssh/authorized_keys é auditável.
- ☐ **M** Acesso a secrets é auditado: log de quem leu o quê e quando (CloudTrail, Vault audit log).
- ☐ **S** Rotação automática mensal de secrets críticos com testes de fallback antes de promover a nova versão.
- ☐ **S** Curto-circuito: se um secret foi exposto em log, alarme dispara em < 5 minutos via SIEM.
- ☐ **S** Chaves criptográficas críticas vivem em KMS/HSM, não em arquivos no disco da instância.
- ☐ **S** Política de "no plaintext at rest" — disco e backup criptografados via KMS gerenciado.
- ☐ **S** Web hooks externos usam HMAC com signing secret rotacionado, não "security through obscurity".
- ☐ **S** Dynamic secrets (Vault) ao invés de credenciais estáticas para banco onde possível.
- ☐ **S** Princípio do menor privilégio: cada secret só pode o que precisa, escopo testado periodicamente.
- ☐ **S** Plano de incidente de vazamento de secret: detectar, revogar, rotacionar, comunicar — em runbook escrito.
- ☐ **S** "Break glass" credentials separadas, em cofre físico/digital fora do sistema, para uso emergencial auditado.
- ☐ **S** Honey tokens espalhados pelo código — credencial falsa que dispara alerta se usada.

⚙️ COMANDOS & FERRAMENTAS

```
# Escaneia o histórico inteiro do repo, todos os branches
$ gitleaks detect --source . --verbose --redact

# TruffleHog — só vulnerabilidades verificáveis
$ trufflehog filesystem . --only-verified

# AWS keys no histórico + pre-commit hook
$ git log --all -S "AKIA" --oneline
$ brew install gitleaks && gitleaks install hooks
```

Como corrigir — antes & depois

✗ Antes: secret hardcoded

```
module.exports = {
  database: "postgres://admin:S3nh4@db.prod/app",
  stripe_key: "sk_live_51HxK...",
  jwt_secret: "meu-segreto-123"
};
```

✓ Depois: env var + validação no boot

```
const required = ['DATABASE_URL', 'STRIPE_KEY', 'JWT_SECRET'];
required.forEach(k => { if (!process.env[k]) throw new Error(`missing ${k}`); });
module.exports = {
  database: process.env.DATABASE_URL,
  stripeKey: process.env.STRIPE_KEY,
  jwtSecret: process.env.JWT_SECRET
};
```

Logs com PII redaction (Pino)

```
const logger = pino({
  redact: {
    paths: ['req.headers.authorization', 'req.headers.cookie',
      '*.password', '*.cardNumber', '*.cpf'],
    censor: '[REDACTED]'
  }
});
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente já trocou todas as keys uma vez — rotação é processo, não evento
- " Nosso secret manager é o .env mesmo — falha grave de classificação
- " Só os devs sêniores têm acesso aos secrets — princípio do menor privilégio violado
- " Não dá pra rotacionar agora, ia quebrar tudo — secret virou single point of failure
- " A gente removeu o secret do código em 2022 — mas e o histórico do Git?

— DEPENDÊNCIAS & SUPPLY CHAIN

CVE é o ponto de partida, não o fim

O QUE É

Cada `npm install`, `pip install` ou `go get` traz centenas de pacotes de terceiros pra dentro do seu binário em produção. A maioria das equipes audita só o que tem CVE público — e ignora typosquatting, dependency confusion e o vetor mais perigoso: o pacote legítimo que foi **comprometido depois** que você instalou.

POR QUE IMPORTA PRO NEGÓCIO

O caso `event-stream` (2018): pacote npm com 2 milhões de downloads semanais foi vendido pelo mantenedor original e recebeu código malicioso. Roubava chaves privadas de carteiras Bitcoin. Mais recentes: `ua-parser-js` (2021), `colors.js` (2022), pacotes do `Polyfill.io` (2024). O atacante moderno não invade você — ele invade quem você confia.

Checklist · 27 verificações

- ☐ **B** `npm audit` / `pip-audit` / `go vet` roda em todo CI; build falha em **high/critical**.
- ☐ **B** Lockfile (`package-lock.json`, `poetry.lock`, `go.sum`) está commitado e é fonte da verdade.
- ☐ **B** `npm install` em CI usa `npm ci` com lockfile imutável, não `npm i`.
- ☐ **B** Dependabot/Renovate habilitado para PRs automáticos de bump.
- ☐ **B** Pacotes deprecated foram identificados e plano de migração existe.
- ☐ **B** Dependências de dev (testes, lint) separadas de runtime — não vão pro container final.
- ☐ **M** Snyk, GitHub Advanced Security ou Socket.dev rodando em todo PR, com bloqueio.
- ☐ **M** Política de "no install scripts" para pacotes não auditados (`--ignore-scripts`).
- ☐ **M** Allowlist de licenças configurada — bloqueia GPL, AGPL e desconhecidas se você é SaaS proprietário.
- ☐ **M** Verificação anti-typosquatting: `npm-name-validator` ou similar antes de adicionar dep.
- ☐ **M** Pacotes internos publicados em registry privado (Verdaccio, Artifactory, GitHub Packages).
- ☐ **M** Scope NPM com `@empresa/` registrado para impedir dependency confusion.
- ☐ **M** Política contra "latest" — versões pinadas com `~` ou `^` conscientes, ou exatas em libs críticas.
- ☐ **M** Container base image atualizada nos últimos 30 dias — Alpine/Distroless preferidos.
- ☐ **M** Trivy ou Gype escaneia containers no build, com gate de `critical=0`.
- ☐ **M** Política de descontinuação: dep sem update em 18 meses entra em revisão obrigatória.
- ☐ **M** Forks internos de libs críticas mantidos, com plano de tomar manutenção se upstream cair.
- ☐ **S** SBOM gerado em todo build (CycloneDX ou SPDX) e armazenado por >1 ano.
- ☐ **S** SBOM publicado junto com release (atestado de procedência verificável).
- ☐ **S** Sigstore/cosign verificando assinaturas de pacotes e imagens base.
- ☐ **S** Scorecard (OSSF) rodando contra dependências críticas — score mínimo definido.
- ☐ **S** Política para "package age": pacotes < 90 dias são auditados manualmente.
- ☐ **S** SLSA Level 2+ no pipeline de build dos pacotes críticos.
- ☐ **S** Análise estática de dependências transitivas, não só diretas.
- ☐ **S** Honey package interno publicado pra detectar tentativas de dependency confusion.
- ☐ **S** Runbook escrito de "supply chain incident" (com exemplos de `event-stream`, `ua-parser-js`).
- ☐ **S** Comitê trimestral revisa top-10 deps por blast radius.

⚙️ COMANDOS & FERRAMENTAS

```
# Audit de dependências (Node)
$ npm audit --audit-level=high
$ npm audit fix --force # cuidado: pode quebrar API

# Snyk — análise profunda + transitivas
$ snyk test --severity-threshold=high
$ snyk monitor # envia pro dashboard

# Container scanning
$ trivy image minha-app:latest --severity CRITICAL,HIGH
$ gype minha-app:latest

# SBOM (CycloneDX)
$ cyclonedx-npm --output-file sbom.json

# Verificar assinatura (cosign)
$ cosign verify --key cosign.pub minha-app:latest
```

Anti-typosquatting na prática

Atacante registra colorss, lodahs, request-promise-native2. Mata um errinho de digitação e você instala malware.

```
# 1. Configure scope para deps internos
{
  "name": "@empresa/api-client",
  "publishConfig": { "registry": "https://npm.empresa.com" }
}

# 2. Bloqueie install scripts em CI
$ npm ci --ignore-scripts
```

Detectando supply chain comprometido

Sinais de que um pacote foi comprometido:

- ☐ Mudança recente de mantenedor; install script novo numa minor version.
- ☐ Spike anômalo de tamanho do pacote sem changelog correspondente.
- ☐ Network call em pacote que historicamente não fazia.
- ☐ Repo arquivado / sem atividade nos últimos 6 meses.

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente roda npm audit, tá tudo OK — você só vê 30% do risco real
- " Nosso lockfile não tá no Git porque dá conflito — você não tem builds reprodutíveis
- " Esse pacote não tem CVE — CVE é evento, comprometimento é processo
- " A gente confia no mantenedor — confiança não é controle
- " Atualizar quebra a build — débito técnico virando débito de segurança

CÓDIGO & LÓGICA DE APLICAÇÃO

SQL injection é só o começo

O QUE É

Vulnerabilidades no código da aplicação — onde o desenvolvedor confia em dado de usuário sem validar, autoriza ações sem checar permissão, ou aplica criptografia errada. OWASP Top 10 mudou: **Broken Access Control** é #1 desde 2021, e BOLA (mexer em ID na URL) é vetor #1 de bug bounty público em APIs.

POR QUE IMPORTA PRO NEGÓCIO

BOLA num e-commerce vira "qualquer cliente vê pedido de qualquer outro cliente" — vazamento massivo de PII. SSRF numa API que processa imagens deixa atacante ler metadata de instância EC2 (e roubar credenciais IAM). Mass assignment em POST /users permite criar admin com um {"role": "admin"}.

Checklist · 30 verificações

- ☐ **B** Todo input do usuário é validado por schema (Zod, Joi, Pydantic, class-validator).
- ☐ **B** Queries SQL usam **prepared statements**, jamais concatenação de strings.
- ☐ **B** Output em HTML usa template engine que escapa por padrão (React, Vue, Jinja2, ERB).
- ☐ **B** Autenticação é checada em todo endpoint (middleware), não rota a rota.
- ☐ **B** Senha é hashada com bcrypt/argon2/scrypt — nunca MD5, SHA1, SHA256 puro.
- ☐ **B** Erros não vazam stack trace pro cliente em produção.
- ☐ **B** Endpoints retornam 401/403/404 corretos — não 500 com detalhes.
- ☐ **M** Autorização (BOLA): cada acesso a recurso checa **ownership** — usuário X só vê dados de X.
- ☐ **M** Endpoints administrativos têm checagem de role (BFLA), não só autenticação.
- ☐ **M** Mass assignment bloqueado: DTOs explícitos definem o que o cliente pode enviar.
- ☐ **M** Upload de arquivo valida MIME real (magic bytes), não só extensão.
- ☐ **M** Upload tem limite de tamanho, anti-vírus (ClamAV) e armazenamento fora da raiz da web.
- ☐ **M** SSRF: requests para URLs externas têm allowlist de domínios; bloqueia ranges privados (169.254.169.254, 127.0.0.0/8, 10.0.0.0/8).
- ☐ **M** XXE desabilitado em parsers XML — entidades externas bloqueadas.
- ☐ **M** Path traversal: nomes de arquivo do usuário passam por path.basename() + allowlist.
- ☐ **M** Race conditions críticas usam transação + lock pessimista ou idempotency key.
- ☐ **M** JWT valida alg esperado explicitamente — rejeita alg: none e alg: HS256 se você usa RS256.
- ☐ **M** Tokens (reset password, magic link) são single-use, expiram em < 15 min, gerados com CSPRNG.
- ☐ **M** Comparação de strings sensíveis usa crypto.timingSafeEqual.
- ☐ **M** Logs registram tentativas de acesso negadas com IP/user-agent.
- ☐ **M** Deserialização confiável: nada de pickle.loads, unserialize() em input não-confiável.
- ☐ **S** SAST (Semgrep, CodeQL, SonarQube) rodando em todo PR com gates configurados.
- ☐ **S** Regras customizadas de SAST para padrões internos (ex: "todo controller deve estender BaseAuth").
- ☐ **S** Fuzz testing (libFuzzer, Atheris) em parsers e funções críticas.
- ☐ **S** Threat modeling escrito (STRIDE) para cada domínio crítico.
- ☐ **S** Code review obrigatório com checklist de segurança em features sensíveis.
- ☐ **S** Privacy by design: PII só é coletada quando essencial, com data minimization.
- ☐ **S** Anti-fraude: rate limit por usuário, device fingerprint, behavior analysis.
- ☐ **S** Audit log de ações sensíveis com correlation ID, immutable storage.

⚙️ COMANDOS & FERRAMENTAS

```
# Semgrep - SAST com regras prontas
$ semgrep --config=p/owasp-top-ten --config=p/security-audit

# CodeQL · Bandit (Python) · gosec (Go) · OWASP ZAP (DAST)
$ codeql database analyze db --format=sarif-latest
$ bandit -r src/ -ll
$ zap-cli quick-scan https://staging.app.com
```

BOLA na prática — antes & depois

✗ Antes: confia no ID da URL

```
app.get('/api/orders/:id', async (req, res) => {
  const order = await Order.findById(req.params.id);
  res.json(order); // 🚩 BOLA
});
```

✓ Depois: ownership check

```
app.get('/api/orders/:id', requireAuth, async (req, res) => {
  const order = await Order.findOne({
    _id: req.params.id, userId: req.user.id // scope forçado
  });
  if (!order) return res.status(404).end();
  res.json(order);
});
```

SSRF mitigation

```
async function isPublicHost(url) {
  const { hostname } = new URL(url);
  const ip = await dns.lookup(hostname);
  return !ip.address.match(/^((127\.|10\.|172\.(1[6-9]|2\d|3[01])\.)|192\.168\.|169\..254\.)$/);
}
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente confia que o frontend não vai mandar isso — atacante não usa o frontend
- " SQL injection já tá resolvido com ORM — ainda dá pra fazer com raw queries
- " Esse endpoint é interno, não precisa autorizar — todo endpoint vira público em algum momento
- " Hash MD5 é mais rápido — e por isso é mais fácil de quebrar com GPU
- " A gente loga tudo, qualquer auditoria a gente acha — incluindo o token do usuário

CORS é uma linha. Faltam outras 8

O QUE É

Camada de proteção do navegador: headers HTTP que dizem ao browser **o que ele pode e não pode fazer** com o seu site. CSP bloqueia XSS, HSTS força HTTPS, SameSite cookies bloqueiam CSRF, rate limit bloqueia brute force. Custa zero implementar, ainda é a maioria dos sites em produção que ignora.

POR QUE IMPORTA PRO NEGÓCIO

Falta de CSP é XSS bem-sucedido virando session hijack. Falta de HSTS é man-in-the-middle no Wi-Fi de aeroporto roubando login. Falta de rate limit é brute force resolvendo a senha do admin em 6 horas, custo zero pro atacante.

Checklist · 26 verificações

- ☐ **B** HTTPS forçado em todas as rotas, redirect 301 de HTTP.
- ☐ **B** Header `Strict-Transport-Security` com `max-age=31536000`; `includeSubDomains`.
- ☐ **B** `X-Content-Type-Options`: `nosniff` em todas as respostas.
- ☐ **B** `X-Frame-Options`: `DENY` ou `CSP frame-ancestors` definido.
- ☐ **B** `Referrer-Policy`: `strict-origin-when-cross-origin` ou mais restritivo.
- ☐ **B** Cookies de sessão com `Secure`, `HttpOnly` e `SameSite=Lax` ou `Strict`.
- ☐ **B** CORS configurado com allowlist explícita de origins, jamais `*` com credentials.
- ☐ **B** Rate limit em endpoints de login, signup e password reset (5–10 req/min).
- ☐ **M** CSP completo com nonces para inline scripts, sem `unsafe-inline` nem `unsafe-eval`.
- ☐ **M** CSP `report-uri` ou `report-to` configurado, alertas no SIEM.
- ☐ **M** CSRF protection em forms — token sincronizado ou double submit cookie.
- ☐ **M** SubResource Integrity (SRI) em scripts/CSS de CDN externo.
- ☐ **M** Rate limit por IP + por usuário autenticado, com bucket diferente para endpoints caros.
- ☐ **M** WAF (Cloudflare, AWS WAF, Imperva) em frente à origin.
- ☐ **M** `Permissions-Policy` restringe acesso a câmera, microfone, geolocation só onde necessário.
- ☐ **M** `Cross-Origin-Opener-Policy` (COOP) e `Cross-Origin-Embedder-Policy` (COEP) configurados se app é sensível.
- ☐ **M** Domínio adicionado ao HSTS preload list (hstspreload.org).
- ☐ **M** Cookies sensíveis com prefix `__Host-` ou `__Secure-`.
- ☐ **M** Bot detection (hCaptcha, Turnstile) em endpoints públicos críticos.
- ☐ **M** TLS 1.3 obrigatório, TLS 1.0 e 1.1 desabilitados, ciphers forward-secrecy only.
- ☐ **M** Certificado monitorado — alerta com 30 dias de antecedência.
- ☐ **S** CSP em modo enforce, não só report-only, com tracking de violações.
- ☐ **S** Trusted Types habilitado para mitigar DOM XSS.
- ☐ **S** Anti-bot avançado (PerimeterX, DataDome) em endpoints de e-commerce/login.
- ☐ **S** Certificate Transparency monitoring — alerta se cert do seu domínio for emitido por terceiros.
- ☐ **S** CAA record no DNS restringe quais CAs podem emitir cert pro seu domínio.

⚙️ COMANDOS & FERRAMENTAS

```
# Auditoria rápida de headers · Mozilla Observatory · SSL Labs · testssl.sh
$ curl -I https://app.com | grep -iE "strict|x-frame|csp|referrer"
$ https://observatory.mozilla.org/analyze/app.com
$ https://www.ssllabs.com/ssltest/analyze.html?d=app.com
$ testssl.sh --full https://app.com
```

Headers ideais (Express + Helmet)

```
const helmet = require('helmet');
app.use(helmet({
  contentSecurityPolicy: { directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", (req, res) => `'nonce-${res.locals.nonce}'`],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "data:", "https://cdn.app.com"],
    frameAncestors: ["'none'"],
    reportUri: "/csp-report"
  }},
  hsts: { maxAge: 31536000, includeSubDomains: true, preload: true },
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' }
}));
```

Cookie de sessão seguro

```
res.cookie('__Host-session', sessionId, {
  httpOnly: true, secure: true, sameSite: 'lax',
  path: '/', maxAge: 15 * 60 * 1000
});
```

Rate limit (express-rate-limit + Redis)

```
const loginLimiter = rateLimit({
  store: new RedisStore({ client: redis }),
  windowMs: 15 * 60 * 1000, max: 5,
  keyGenerator: req => `${req.ip}:${req.body.email}`
});
app.post('/login', loginLimiter, loginHandler);
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente tem CORS configurado, tá protegido — CORS não é proteção, é permissão
- " CSP quebra muita coisa, deixa report-only — report-only nunca vai pra enforce
- " Login não tem captcha porque atrapalha conversão — até a primeira invasão
- " A gente usa SSL — TLS 1.0 com cifra fraca também é "SSL"
- " CSRF? Estamos numa SPA, não precisa — endpoints state-changing precisam, sim

Login funciona ≠ login **seguro**

O QUE É

Autenticação prova quem você é. Autorização decide o que você pode fazer. Os dois precisam estar certos. JWT mal validado, refresh token sem rotação, MFA opcional, OAuth misconfigurado — qualquer um deles vira "atacante vira admin em uma requisição".

POR QUE IMPORTA PRO NEGÓCIO

Conta de admin comprometida significa acesso total à base de dados, sem alarme, sem rastro. JWT com alg: none aceito permite atacante forjar qualquer identidade. MFA bypass é o que separa "tentaram hackear" de "vazamos 50k clientes" no relatório do IBM Cost of Breach.

Checklist · 28 verificações

- ☐ **B** Senhas com mínimo 12 caracteres, banidas as top 10k mais comuns (zxcvbn ou Have I Been Pwned).
- ☐ **B** Bcrypt com cost ≥ 12 ou Argon2id com params recomendados pela OWASP.
- ☐ **B** JWT valida explicitamente o alg esperado, rejeita none.
- ☐ **B** JWT secrets/keys diferentes entre ambientes; chaves RS256 para apps multi-serviço.
- ☐ **B** Tokens de session/refresh têm exp definido — máximo 30 dias para refresh.
- ☐ **B** Logout invalida sessão server-side, não só limpa cookie.
- ☐ **B** Login enumeration mitigado — mensagem genérica "credenciais inválidas".
- ☐ **M** MFA disponível para todos os usuários (TOTP, WebAuthn).
- ☐ **M** MFA **obrigatório** para usuários admin/staff.
- ☐ **M** Refresh token rotation: cada uso invalida o anterior, detecta replay.
- ☐ **M** Token reuse detection: se refresh token antigo for usado, revoga toda a sessão.
- ☐ **M** Email de notificação em mudanças sensíveis: senha, email, MFA, novo dispositivo.
- ☐ **M** Account lockout temporário após N falhas, com unlock timer ou suporte.
- ☐ **M** Reset password usa token single-use que expira em < 30 min.
- ☐ **M** OAuth: state param obrigatório, redirect URI em allowlist exata (não wildcard).
- ☐ **M** OAuth: PKCE habilitado para clients públicos (mobile, SPA).
- ☐ **M** Sessões listáveis pelo usuário, com opção de "encerrar todas as outras".
- ☐ **M** RBAC com roles definidas, não permissões espalhadas no código.
- ☐ **M** Audit log de login: quando, IP, user-agent, sucesso/falha, com geolocation.
- ☐ **M** Detecção de impossible travel — login em SP às 14h e em Tóquio às 14h05 alarma.
- ☐ **S** WebAuthn / Passkeys disponível como segundo fator (ou primário, passwordless).
- ☐ **S** ABAC para autorização contextual (location, time, device trust).
- ☐ **S** Step-up authentication: ações sensíveis (transferência, mudança de admin) pedem MFA novamente.
- ☐ **S** Risk-based authentication: device fingerprint + behavior baseline.
- ☐ **S** Just-in-time access para admin (Teleport, AWS SSO com session limit).
- ☐ **S** Política de revogação de acesso ≤ 24h após desligamento (offboarding automatizado).
- ☐ **S** SCIM provisioning entre IdP (Okta, Azure AD) e apps SaaS.
- ☐ **S** Anti-phishing: notificação em qualquer login com hardware key (hardware-bound).

⚙️ COMANDOS & FERRAMENTAS

```
# Verifica se senhas estão em vazamentos públicos
$ curl https://api.pwnedpasswords.com/range/$(echo -n "minha-senha" | shasum | cut -c1-5)

# Decode e inspeção de JWT (não confie no payload)
$ jwt-cli decode "eyJhbGciOiJIUzI1NiI..."

# OAuth flow tester
$ oauth-utils verify-pkce --client-id=...
```

JWT seguro — antes & depois

❌ Antes: aceita qualquer alg

```
const decoded = jwt.verify(token, secret); // 🚩 aceita "none"
```

✓ Depois: alg fixo

```
const decoded = jwt.verify(token, publicKey, {
  algorithms: ['RS256'], // fixo, não negociável
  issuer: 'https://auth.app.com',
  audience: 'app-api',
  clockTolerance: 10
});
```

Refresh token rotation com detecção de reuso

```
async function rotateRefreshToken(oldToken) {
  const stored = await db.findToken(oldToken);
  if (!stored || stored.used) {
    // Token reuse detected — atacante tem token antigo
    await db.revokeAllUserSessions(stored?.userId);
    throw new Error('token reuse detected');
  }
  await db.markTokenUsed(oldToken);
  return generateNewTokenPair(stored.userId);
}
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " JWT é stateless, dá pra invalidar não — usar deny list ou revogar a chave de assinatura
- " MFA é chato pro usuário — até o usuário perder a conta e culpar você
- " Nossa role admin pode tudo — princípio do menor privilégio é pra todos
- " A gente confia no token uma vez emitido — refresh tokens precisam de rotação
- " OAuth usa redirect URI com wildcard — open redirect virando token leak

BANCO DE DADOS & PII

SQL injection é o **menor** dos problemas

O QUE É

Banco é onde mora o que dá processo: CPF, cartão, conversa privada, histórico médico. Camada de defesa é múltipla: criptografia em repouso e em trânsito, mascaramento de PII, RLS pra evitar tenant leak, backup que **você efetivamente conseguiu restaurar** num teste cego.

POR QUE IMPORTA PRO NEGÓCIO

Backup que nunca foi testado é "esperança", não estratégia — 90% das empresas só descobrem que o backup tá quebrado quando precisa. Em multitenant SaaS sem RLS, um query errado mostra dados do cliente A pro cliente B. Vazamento de coluna de cartão sem encryption-at-rest é multa PCI-DSS imediata.

Checklist · 25 verificações

- ☐ **B** Conexão ao banco usa TLS sempre, com cert validation.
- ☐ **B** Usuário do banco da app não é root/postgres — tem permissão mínima necessária.
- ☐ **B** Sem credenciais default em nenhum banco (Mongo, Redis, Postgres, MySQL).
- ☐ **B** Banco não está exposto na internet pública — security group / firewall fechado.
- ☐ **B** Backup automatizado, frequência condizente com RPO definido.
- ☐ **B** Encryption at rest habilitado (AWS RDS encryption, transparent data encryption).
- ☐ **M** Backup já foi testado com restore real em ambiente isolado nos últimos 90 dias.
- ☐ **M** RTO e RPO documentados, testados via tabletop exercise.
- ☐ **M** NoSQL injection prevenido — drivers parametrizados, validação de tipo no query builder.
- ☐ **M** Row Level Security (RLS) ativa em multitenant SaaS, com testes automatizados.
- ☐ **M** Column-level encryption em CPF, cartão, dados médicos (pgcrypto, AWS KMS).
- ☐ **M** Tokenização ou format-preserving encryption para PCI scope.
- ☐ **M** Audit log de queries em tabelas sensíveis (pg_audit, MySQL audit plugin).
- ☐ **M** Connection pool limita conexões; statement_timeout previne queries pesadas.
- ☐ **M** Read replica usada por workloads de leitura, isolando produção transacional.
- ☐ **M** Snapshots de backup armazenados em conta/região separada (cross-account).
- ☐ **M** Migrations versionadas, com rollback testado, jamais aplicadas manualmente em prod.
- ☐ **S** RLS testada com fuzz: ataques cross-tenant impossíveis por design.
- ☐ **S** Data masking automático em ambientes não-prod (Bytebase, Aiven).
- ☐ **S** DLP monitora exfiltração: query que retorna > X linhas em < Y segundos alarma.
- ☐ **S** Immutable / WORM backup pra ransomware (S3 Object Lock, Veeam).
- ☐ **S** Honey table — tabela com PII falsa que dispara alerta se for lida.
- ☐ **S** Data classification: cada coluna tem tag (public/internal/confidential/PII/PCI).
- ☐ **S** Retention policy automatizada: dados expiram conforme LGPD art. 16.
- ☐ **S** Database firewall (jSonar, IBM Guardium) bloqueia query patterns suspeitos.

⚙️ COMANDOS & FERRAMENTAS

```
# Postgres — checa conexões SSL ativas
$ psql -c "SELECT ssl, version, client_addr FROM pg_stat_ssl JOIN pg_stat_activity USING(pid);"

# Encontra colunas com PII potencial · Teste RLS · Backup test
$ psql -c "SELECT table_name, column_name FROM information_schema.columns WHERE column_name ~* '(cpf|card|email|phone|ssn)';"
$ psql -c "SET app.tenant_id = 'tenant_a'; SELECT * FROM orders WHERE tenant_id = 'tenant_b';"
$ pg_restore -d test_restore backup.dump && psql test_restore -c "SELECT count(*) FROM users;"
```

Row Level Security (Postgres)

```
-- Multi-tenant: cada query enxerga só o próprio tenant
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
-- Em cada conexão, app seta o tenant atual
SET app.tenant_id = 'a4e8c1...';
```

Column-level encryption

```
CREATE EXTENSION pgcrypto;
INSERT INTO users (id, cpf_encrypted) VALUES (
  uuid_generate_v4(),
  pgp_sym_encrypt('12345678900', current_setting('app.encryption_key'))
);
-- Lê descriptografado (só com a chave do KMS)
SELECT id, pgp_sym_decrypt(cpf_encrypted, ...) FROM users;
```

Backup test runbook

- ☐ 1. Pega o backup mais recente do storage cross-account.
- ☐ 2. Restaura em instância staging vazia (timer começa).
- ☐ 3. Roda smoke tests e compara checksums de tabelas-chave.
- ☐ 4. Documenta tempo total + qualquer anomalia. RTO real registrado.

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente faz backup todo dia — quando foi a última vez que testou o restore?
- " Encryption at rest tá ON na AWS — e a chave KMS, quem controla?
- " SQL injection? Usamos ORM — raw queries continuam aceitando string concat
- " RLS é overhead — multitenant sem RLS é tenant leak esperando acontecer
- " CPF está em texto plano, mas o banco é privado — privado até alguém vaziar credencial

A nuvem não é segura por padrão. **É configurável**

O QUE É

Modelo de responsabilidade compartilhada: o provedor protege a infra física, você protege o que sobe nela. S3 público, IAM com Action: *, SG aberto pra 0.0.0.0/0 — todos são default ou erro humano que custa caro. **75% dos incidentes em cloud** em 2024 vieram de misconfiguration, não de exploit.

POR QUE IMPORTA PRO NEGÓCIO

S3 público com PII de clientes vira manchete em 24h. Role IAM com *: * que vaza vira "atacante criou 200 instâncias EC2 minerando crypto". Container rodando como root + exploit no kernel = host inteiro comprometido, possivelmente toda sua infra.

Checklist · 28 verificações

- ☐ **B** Nenhum bucket S3/GCS público sem motivo documentado e assinado.
- ☐ **B** Block Public Access habilitado por padrão na conta AWS.
- ☐ **B** Conta root da AWS com MFA, sem access keys ativas, alerta em qualquer login.
- ☐ **B** CloudTrail habilitado em todas as regiões, logs em conta separada (audit account).
- ☐ **B** Security Groups sem 0.0.0.0/0 em portas administrativas (22, 3389, 5432, etc).
- ☐ **B** VPC privada para recursos internos; bastion host ou SSM Session Manager para SSH.
- ☐ **B** Imagens base atualizadas (Alpine 3.18+, Distrosless, Wolfi).
- ☐ **M** IAM com least privilege real — sem Action: * em produção.
- ☐ **M** IAM Access Analyzer habilitado, findings revisados semanalmente.
- ☐ **M** Roles assumidas via IRSA (EKS) ou Workload Identity (GKE), sem credentials estáticas em pods.
- ☐ **M** AWS Config / GCP Asset Inventory tracking drift de configuração.
- ☐ **M** Multi-account strategy: produção, staging, audit, security em contas separadas com SCP.
- ☐ **M** GuardDuty / Cloud Armor habilitado, alertas pra Slack/PagerDuty.
- ☐ **M** Terraform/IaC obrigatório para mudanças — sem clickops em prod.
- ☐ **M** Checkov, Tfsec ou tfint em todo PR de infra.
- ☐ **M** Containers não rodam como root (USER 1000 no Dockerfile).
- ☐ **M** Pod Security Standards configurado (restricted) no Kubernetes.
- ☐ **M** Network policies restringem tráfego pod-to-pod (Calico, Cilium).
- ☐ **M** Service mesh (Istio, Linkerd) com mTLS entre serviços críticos.
- ☐ **M** Secrets em K8s usam external secret operator + Secrets Manager, não secrets nativos em texto.
- ☐ **S** CSPM (Wiz, Prowler, Steampipe) rodando contra a conta inteira.
- ☐ **S** CIS Benchmark compliance score acompanhado mensalmente.
- ☐ **S** Runtime security (Falco, Sysdig Secure, Tetragon) detectando comportamento anômalo.
- ☐ **S** Image signing com cosign + admission controller validando assinatura no cluster.
- ☐ **S** Just-in-time access (Teleport, AWS Identity Center) — sem usuários permanentes.
- ☐ **S** Disaster recovery testado com chaos engineering (Chaos Monkey, AWS FIS).
- ☐ **S** SCP (Service Control Policies) bloqueia regiões/serviços não usados.
- ☐ **S** Workload identity federation entre clouds — sem chaves cross-cloud.

⚙️ COMANDOS & FERRAMENTAS

```
# Prowler — auditoria CIS Benchmark da AWS
$ prowler aws --severity high,critical

# Buckets S3 públicos · Tfsec / Checkov · Trivy
$ aws s3api list-buckets --query 'Buckets[].Name'
$ tfsec . && checkov -d . --framework terraform
$ trivy image --scanners vuln,config,secret minha-app:v1
```

IAM least privilege — antes & depois

✗ Antes: god mode

```
{ "Effect": "Allow", "Action": "*", "Resource": "*" }
```

✓ Depois: scoped

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::uploads-prod/users/${aws:userid}/*",
  "Condition": { "Bool": { "aws:SecureTransport": "true" } }
}
```

Dockerfile seguro

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM gcr.io/distroless/nodejs20-debian12
COPY --from=builder /app/node_modules ./node_modules
COPY --chown=nonroot:nonroot . .
USER nonroot:nonroot      # UID 65532, sem shell
CMD ["server.js"]
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- A AWS é segura por padrão — segurança compartilhada, lembra?
- Tudo em uma conta só, mais simples — explosion radius enorme
- A gente faz mudança direto no console pra agilizar — auditoria zero, drift garantido
- Container roda como root, é só dev — escape de container vira problema sério
- Acesso permanente é mais prático — JIT é o padrão moderno por uma razão

É aqui que o atacante **moderno** entra

O QUE É

Pipeline de build é onde a sua aplicação se transforma em artefato em produção. Em 2024, mais ataques vieram pelo pipeline (SolarWinds, Codecov, GitHub Actions exploits) do que pelo código diretamente. Atacante moderno vai pra onde tem confiança implícita — secrets do CI, runners self-hosted, scripts de PR.

POR QUE IMPORTA PRO NEGÓCIO

Pull request malicioso de qualquer fork do GitHub pode ler seus secrets se o workflow estiver com `pull_request_target` mal configurado. Build artifact substituído entre build e deploy quebra o "código revisado é o código em produção". Sem SLSA, você não **consegue provar** que o que tá rodando é o que você commitou.

Checklist · 26 verificações

- ☐ **B** Branch protection no main: PR obrigatório, review, status checks.
- ☐ **B** Force-push e branch deletion bloqueados em branches de produção.
- ☐ **B** Signed commits obrigatórios no main (GPG ou SSH signing).
- ☐ **B** Secrets do CI/CD em vault gerenciado, jamais em repo.
- ☐ **B** Workflows não fazem echo de secrets em logs.
- ☐ **B** CODEOWNERS configurado em paths sensíveis.
- ☐ **M** Workflows pin'am actions por SHA, não por tag (@v4 é mutável).
- ☐ **M** `pull_request_target` usado com cuidado extremo, sem checkout do PR sem revisão.
- ☐ **M** GITHUB_TOKEN com permissão **read** por padrão, write apenas onde necessário.
- ☐ **M** Secrets segregados por ambiente (dev/staging/prod), não compartilhados.
- ☐ **M** Required environments com aprovação manual antes de deploy em produção.
- ☐ **M** Self-hosted runners isolados em rede própria, ephemeral por job.
- ☐ **M** Build artifacts assinados com cosign, deploy só aceita assinados.
- ☐ **M** Dependabot/Renovate configurado pra actions também (não só deps de código).
- ☐ **M** Secret scanning ativo no repo (push protection ligado).
- ☐ **M** Workflows aprovados explicitamente para forks; padrão é "require approval".
- ☐ **M** Audit log do GitHub/GitLab exportado pro SIEM.
- ☐ **M** Deploy keys / PATs revogados após uso ou rotacionados periodicamente.
- ☐ **S** SLSA Level 2+ no pipeline: provenance gerada e verificada.
- ☐ **S** Sigstore + Rekor para transparency log de artefatos.
- ☐ **S** Hermetic builds: sem network, dependências resolvidas antecipadamente.
- ☐ **S** Reproducible builds — mesmo input gera mesmo binário.
- ☐ **S** Two-person review obrigatório para mudanças em workflows e infra crítica.
- ☐ **S** OIDC entre GitHub Actions e cloud — sem long-lived secrets em CI.
- ☐ **S** Tampering detection: alarme se workflow YAML for modificado sem aprovação.
- ☐ **S** Tabletop exercise anual de "comprometimento de pipeline": como detectar e responder.

⚙️ COMANDOS & FERRAMENTAS

```
# Audita workflows · Lista PATs ativos
$ gh-actions-audit --org minha-empresa
$ gh api /user/tokens

# Action mutável (RUIM): actions/checkout@v4
# Pin por SHA (BOM):
- uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11

# Verifica provenance SLSA
$ slsa-verifier verify-artifact --provenance-path provenance.json \
  --source-uri github.com/empresa/app artifact.tar.gz
```

Workflow seguro — antes & depois

✗ Antes: vulnerável a PR injection

```
on: pull_request_target # tem secrets!
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${ github.event.pull_request.head.sha }
      - run: npm install && npm test // roda código do PR
```

✓ Depois: jobs separados, action pinada por SHA

```
on: pull_request # sem secrets pro fork
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@b4ffde65f46336ab88eb53be...
      - run: npm ci --ignore-scripts && npm test
```

⚠️ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " Usamos @latest ou @v4 nas actions — tag é mutável, atacante pode trocar
- " CI roda código do PR direto, é mais conveniente — porta aberta pra fork malicioso
- " Tem um PAT genérico que todo mundo usa — owner desse PAT é o quê?
- " Self-hosted runner roda no laptop do dev — você confia tudo no laptop?
- " Deploy é manual mesmo — sem auditoria, sem rastro, sem rollback automatizado

LGPD não é "tem termo de uso?"

O QUE É

Conformidade regulatória traduzida em controles técnicos. LGPD (Brasil), GDPR (UE), PCI-DSS (cartão), ISO 27001 (segurança da informação), SOC 2 (operação de SaaS B2B). Não é "burocracia do jurídico" — é o que define se você pode operar em determinados mercados e o tamanho da multa quando algo der errado.

POR QUE IMPORTA PRO NEGÓCIO

LGPD: até **2% do faturamento** ou R\$ 50 milhões por infração. SOC 2 Type II é pré-requisito pra vender pra empresas grandes. PCI-DSS sem compliance significa não processar cartão direto — você vai pra Stripe, perde 3% de margem. ISO 27001 abre licitações públicas e contratos enterprise.

Checklist · 27 verificações

- ☐ **B** Inventário de dados pessoais tratados (registro das operações de tratamento, art. 37 LGPD).
- ☐ **B** Base legal definida para cada tratamento (consentimento, execução de contrato, legítimo interesse, etc).
- ☐ **B** Política de Privacidade publicada, em linguagem acessível, atualizada.
- ☐ **B** Termo de Uso separado da Política de Privacidade.
- ☐ **B** Cookie banner com opt-in granular para cookies não-essenciais.
- ☐ **B** Canal de contato pra titular exercer direitos (e-mail dedicado, formulário).
- ☐ **B** DPO (Encarregado) designado e divulgado se há tratamento sistemático.
- ☐ **M** 8 direitos do titular implementados tecnicamente — ver Apêndice D.
- ☐ **M** DPIA (RIPD) elaborada para tratamentos de alto risco.
- ☐ **M** DPA (acordo de tratamento) com todo fornecedor SaaS que processa dados pessoais.
- ☐ **M** Sub-operadores listados publicamente.
- ☐ **M** Transferência internacional com cláusulas contratuais ou país com nível adequado.
- ☐ **M** Retention policy automatizada — dados deletados conforme art. 16 LGPD.
- ☐ **M** Anonymization/pseudonymization aplicada onde possível.
- ☐ **M** Breach response plan: comunicação à ANPD em < 72h e aos titulares afetados.
- ☐ **M** Minor consent: tratamento de dados de criança/adolescente com consentimento dos pais.
- ☐ **M** Privacy by Design documentado em features novas.
- ☐ **M** Treinamento anual em LGPD para equipes de produto, dev e suporte.
- ☐ **S** ISO 27001 (ou em processo de certificação) com SoA documentado.
- ☐ **S** SOC 2 Type II auditado anualmente — controles em CC, A, P, C, PI.
- ☐ **S** PCI-DSS aplicado se processa cartão direto, ou tokenização via PSP certificado.
- ☐ **S** NIST CSF (Cybersecurity Framework) usado como mapa de maturidade.
- ☐ **S** CIS Controls v8 implementadas e auditadas.
- ☐ **S** Trust Center público com SOC reports (sob NDA), DPA modelo, security overview.
- ☐ **S** Insurance: cyber liability + tech E&O dimensionada pela ARR.
- ☐ **S** Vendor risk management formal: questionário, score, revisão anual.
- ☐ **S** Ethical AI: viés, transparência e accountability documentados se usa IA com decisão automatizada.

Mapa de frameworks por tipo de negócio

SAAS B2B BRASILEIRO

Mínimo: LGPD compliance + DPA pronta + cookie banner.

Pra crescer: SOC 2 Type II + ISO 27001.

Enterprise: + Trust Center público, cyber insurance.

E-COMMERCE / FINTECH

Obrigatório: LGPD + PCI-DSS (ou tokenização via PSP) + Bacen Resolução 4.658 se for instituição financeira.

Recomendado: ISO 27001, controles de prevenção a fraude.

HEALTHTECH

Crítico: LGPD + dados sensíveis (art. 11) + ANS/CFM regras setoriais. Se atende EUA: HIPAA. Se UE: GDPR + medical device regulation.

VENDE PARA GOVERNO BRASILEIRO

Geralmente exigido: ISO 27001 + LGPD + comprovações específicas do edital. Marco Civil + leis setoriais aplicáveis.

Como a equipe do produto se organiza

- **Privacy review** em todo épico que toque dado pessoal — checklist no PR template.
- **DPIA leve** (1 página) para features novas; DPIA completa para tratamento de alto risco.
- **Data classification** em todo schema migration: tag obrigatória de sensibilidade.
- **Quarterly access review:** quem ainda precisa do acesso que tem? Revoga o resto.
- **Annual policy review:** política de privacidade, retenção, incident response.

Templates práticos

Os templates abaixo estão no **Apêndice C**:

- ☐ Incident response runbook (com timing ANPD)
- ☐ DPIA simplificada (1 página)
- ☐ Script de comunicação a titulares pós-vazamento
- ☐ DPA modelo para fornecedores

⚠ RED FLAGS · SINAIS DE EQUIPE IMATURA

- " A gente tem termo de uso atualizado — termo ≠ política de privacidade
- " DPO é o sócio mesmo — conflito de interesse, não tem independência
- " Cookie banner aceita por padrão — opt-in é a regra, não opt-out
- " Vamos fazer LGPD depois do product-market fit — multa não vai esperar PMF
- " Compliance é problema do jurídico — todo controle é técnico antes de ser jurídico

A SKILL

A skill

`claude /security-audit`

O QUE ELA COBRE

A skill automatiza a primeira passagem de auditoria pelas 9 camadas — detecta padrões óbvios, popula um relatório priorizado e indica próximos passos. **Cobertura honesta: ~30%** dos 240+ checks deste documento. Os outros 70% precisam de revisão humana, threat modeling ou pentest.

COBERTURA MAPEADA POR CAMADA

CAMADA	TOTAL CHECKS	AUTO	COBERTURA
01 · Secrets	28	14	~50%
02 · Dependências	27	13	~48%
03 · Código	30	8	~27%
04 · Web Security	26	12	~46%
05 · Auth & AuthZ	28	6	~21%
06 · Banco de Dados	25	7	~28%
07 · Cloud & Infra	28	9	~32%
08 · CI/CD	26	8	~31%
09 · Compliance	27	2	~7%

Por que a cobertura cai em camadas como Auth e Compliance? Porque essas dependem de contexto de negócio que máquina não tem: "esse refresh token é seguro?" depende do modelo de ameaça do produto; "esse tratamento exige DPIA?" depende do impacto potencial pra titulares.

OUTPUT ESPERADO

```
$ claude /security-audit

📁 Auditoria de Segurança — app-prod
  Camadas escaneadas: 9
  Checks executados: 79 / 245

🔴 Critical (3)
  ▶ JWT secret hardcoded em src/auth/config.ts:14
  ▶ Bucket S3 'uploads-prod' está público
  ▶ Endpoint POST /api/admin sem auth middleware

🟡 High (8)
  ▶ Dependência axios@0.21.1 com CVE-2021-3749
  ▶ HSTS ausente no domínio principal
  ▶ ... +6

🟢 Medium (12) • // detalhes em report.md

→ Salvou: .security/audit-2026-05-07.md
```

Como instalar

1 · Clone o template base

```
$ git clone https://github.com/danillodantas/security-audit-skill ~/.claude/skills/security-audit
```

2 · Instale ferramentas locais

```
$ brew install gitleaks trufflehog trivy semgrep  
$ npm install -g snyk
```

3 · Rode no seu projeto

```
$ cd meu-projeto  
$ claude  
> /security-audit
```

Estrutura da skill

```
~/.claude/skills/security-audit/  
├─ SKILL.md           # instruções pro Claude  
├─ checklist/  
│   ├── 01-secrets.md  
│   ├── 02-dependencies.md  
│   └─ ... (9 camadas)  
├─ scripts/  
│   ├── scan-secrets.sh  
│   ├── scan-deps.sh  
│   ├── scan-cloud.sh  
│   └─ ...  
└─ templates/  
    └─ report.md.tpl
```

Limitações conhecidas — seja honesto

- **Não substitui pentest** — não acha vulns que precisam de criatividade humana
- **Falsos positivos existem** — confirme antes de marcar como crítico
- **Não testa runtime** — vulns que só aparecem com tráfego real (race conditions, IDOR contextual) escapam
- **Compliance fica fora** — auditoria LGPD/SOC2 exige documentação humana
- **Bias por linguagem** — funciona melhor em Node/Python/Go; ecossistemas menores tem menos regras

Como interpretar o output

Critical: bloqueia merge / deploy. Resolva antes de qualquer coisa.

High: trata na próxima sprint. Documenta exceção temporária se for postergar.

Integrando no fluxo do time

No CI (GitHub Actions)

```
name: Security Audit
on:
  pull_request:
    branches: [main]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@b4ffde65...
      - name: Run security-audit
        run: ./scripts/security-audit.sh --gate=critical
      - uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: report.sarif
```

Como red flag de PR

Configure o job pra falhar se houver qualquer **critical**. Reviewer não merge enquanto não estiver verde.

Mensal — auditoria full

Rode com flag `--all` mensalmente. Compare com baseline anterior, anote mudanças, atualiza roadmap de segurança.

Pra onde a skill aponta

Cada finding linka pra documentação correspondente neste guia + sugere ferramenta para fix profundo. Exemplo:

- **[CRITICAL]** Secret hardcoded em `src/config.ts:14`
 - > Camada 1 (Secrets) — ver pág. 4–5 do guia
 - > Tool: `gitleaks` + `secret manager`
 - > Como corrigir:
 1. Remova do código fonte
 2. `git filter-repo` pra limpar histórico
 3. Migre pra AWS Secrets Manager / Doppler
 4. Rotacione a credencial vazada
 5. Rode `gitleaks detect` pra confirmar

Quando a skill não é suficiente

Você precisa de auditoria sênior ou pentest profissional quando:

- ☐ Está pra fechar enterprise contract que exige SOC 2 Type II
- ☐ Vai processar volumes maiores de dados sensíveis (saúde, financeiro)
- ☐ Sofreu (ou quase sofreu) incidente — quer entender extensão
- ☐ Recebeu disclosure responsável do pesquisador externo

— CADÊNCIA

Cadência de auditoria por camada

Segurança não é "rodou uma vez". É processo contínuo com cadências diferentes para cada camada. Use esta tabela como mapa anual.

CAMADA	EM PR	MENSAL	TRIM.	ANUAL
01 · Secrets	•	•	•	•
02 · Dependências	•	•	—	•
03 · Código	•	—	•	•
04 · Web Security	•	—	•	•
05 · Auth & AuthZ	•	—	•	•
06 · Banco de Dados	—	•	•	•
07 · Cloud & Infra	•	•	•	•
08 · CI/CD	•	—	•	•
09 · Compliance	—	—	•	•

A CADA PR — BLOQUEIO AUTOMÁTICO EM CI

Skill /security-audit roda automaticamente. SAST + secret scanning + dependency check. Falha imediatamente em **critical**; high vira comentário no PR.

MENSAL — RITMO DO TIME DE PLATAFORMA

Rotação de secrets críticos. Atualização de imagens base. Revisão de logs & alertas. Backup restore test em ambiente isolado. Análise de findings novos do CSPM.

TRIMESTRAL — REVISÃO SÊNIOR

Auditoria humana nas 9 camadas, foco no que mudou no trimestre. Threat modeling de features novas. Quarterly access review (quem ainda precisa do que tem?). Tabletop exercise de incidente.

ANUAL — PENTEST PROFISSIONAL

Pentest externo com black-box + grey-box. Auditoria de SOC 2 / ISO 27001 (se aplicável). Revisão de políticas, treinamento corporativo. Renovação de cyber insurance com base em métricas.

In-house vs profissional externo

FAZ INTERNAMENTE

- ☐ Auditoria automatizada em todo PR
- ☐ Hardening contínuo com checklist
- ☐ Code review com lente de segurança
- ☐ Incident response inicial
- ☐ Backup test, secret rotation, access review

CONTRATA EXTERNO

- ☐ Pentest anual (caixa-preta + caixa-cinza)
- ☐ Threat modeling em arquitetura nova
- ☐ Auditoria SOC 2 / ISO 27001
- ☐ Resposta a incidente grave (forense)
- ☐ Bug bounty managed (HackerOne, BugCrowd)

Sinais de que você precisa parar de improvisar

- ☐ Cliente enterprise pediu SOC 2 e você não tem ninguém pra responder o questionário
- ☐ ANPD entrou em contato sobre denúncia
- ☐ Você descobriu uma vuln depois de pesquisador reportar publicamente
- ☐ O time de TI/dev tá fazendo segurança no tempo livre
- ☐ Não existe runbook escrito de incidente
- ☐ Última auditoria foi "ano passado, mais ou menos"
- ☐ Cyber insurance recusou renovação ou aumentou prêmio em 50%+

Custo de não fazer

Pentest profissional sério: **R\$ 25k a R\$ 80k** dependendo do escopo. Auditoria SOC 2 Type II inicial: **R\$ 100k a R\$ 250k**. Custo médio de breach (IBM 2024) no Brasil: **R\$ 6,75 milhões**. A conta é trivial.

— PRÓXIMOS PASSOS

Como avançar a partir daqui

Ferramentas open-source — por categoria

SECRETS

gitleaks · trufflehog · detect-secrets · Doppler (free tier) · HashiCorp Vault.

DEPENDÊNCIAS

Snyk (free tier) · Dependabot · Renovate · Trivy · Gype · Socket.dev · OSSF Scorecard.

SAST / DAST

Semgrep · CodeQL · Bandit · gosec · OWASP ZAP · Nuclei · SQLMap.

CLOUD / IAC

Prowler · Steampipe · Checkov · Tfsec · Terrascan · KICS · CloudSploit.

CONTAINER / K8S

Trivy · Falco · Tetragon · kube-bench · kube-hunter · Kyverno · OPA Gatekeeper.

SUPPLY CHAIN

Sigstore (cosign + Rekor) · in-toto · slsa-github-generator · Chainguard Images.

Comunidades para acompanhar

OWASP — Top 10, ASVS, Cheat Sheets, projetos por linguagem

CWE / CVE — Mitre, NVD, GitHub Security Advisories

CIS Controls v8 — base prática de hardening

NIST CSF — framework de maturidade

SANS — newsletters, certificações sérias

Kebs on Security · **DarkReading** · **The Hacker News** — news técnico

— PRÓXIMO PASSO

Quer auditoria sênior nas 9 camadas?

Se este checklist mostrou buracos que você precisa fechar — e fechar com responsabilidade técnica e operacional — o próximo passo é uma conversa.

SCALE PERFORMANCE

Consultoria sênior em segurança aplicada, auditoria das 9 camadas no contexto do seu negócio, threat modeling, implementação de controles. Foco em SaaS B2B brasileiro.

→ Agendamento direto pelo Instagram [@danillodantas](https://www.instagram.com/danillodantas).ia

Mais conteúdo neste tema

- Carrossel completo: [instagram.com/p/DYD1zh1keHL](https://www.instagram.com/p/DYD1zh1keHL)
- Próximas publicações: aprofundamento por camada, casos reais, post-mortem de incidentes públicos

Como aplicar este guia nesta semana

- ☐ **Hoje:** rode gitleaks detect no histórico do seu repositório principal
- ☐ **Esta semana:** ative branch protection + signed commits + Dependabot
- ☐ **Este mês:** Camadas 1, 2, 4 e 7 cobertas no básico
- ☐ **Este trimestre:** threat modeling do produto + DPIA + access review
- ☐ **Este ano:** pentest profissional anual + plano SOC 2 (se for SaaS B2B)



Danillo Dantas
[@danillodantas](https://www.instagram.com/danillodantas).ia

Compartilhe este checklist com quem precisa entender que segurança não é gasto de TI
— é proteção do CNPJ.

Termos & siglas

Os termos que mais aparecem em conversas sobre as 9 camadas. Use como referência cruzada.

BOLA · Broken Object Level Authorization

Permitir que usuário A acesse recurso do usuário B trocando o ID. OWASP API Security #1.

BFLA · Broken Function Level Authorization

Usuário comum acessa endpoint só de admin. Falha de autorização funcional.

SSRF · Server-Side Request Forgery

Atacante força sua app a fazer request pra URL que ele controla — clássico em metadata da AWS (169.254.169.254).

IDOR · Insecure Direct Object Reference

Termo guarda-chuva para BOLA — referenciar objeto direto pelo ID sem checar permissão.

XXE · XML External Entity

Parser XML processando entidades externas — leitura de arquivos do servidor, SSRF.

CSRF · Cross-Site Request Forgery

Site malicioso forja request autenticado em nome do usuário logado em outro site.

XSS · Cross-Site Scripting

Atacante injeta JavaScript que executa no browser de outras vítimas.

RLS · Row Level Security

Banco filtra automaticamente linhas de acordo com identidade do usuário/tenant.

SBOM · Software Bill of Materials

Lista exaustiva de toda dependência, versão e origem de um artefato.

SLSA · Supply-chain Levels for Software Artifacts

Framework Google para garantir integridade entre código e binário em produção. Níveis 1 a 4.

CSP · Content Security Policy

Header HTTP que define quais origens podem executar scripts/imagens/etc na sua página.

HSTS · HTTP Strict Transport Security

CORS · Cross-Origin Resource Sharing

Mecanismo que permite (ou nega) requests entre origens diferentes. Não é proteção, é permissão.

PKCE · Proof Key for Code Exchange

Extensão do OAuth para clientes públicos (mobile, SPA) — mitiga interceptação do auth code.

JWT · JSON Web Token

Token assinado, formato padrão pra autenticação stateless. Atenção a `alg`: none.

MFA · Multi-Factor Authentication

Algo que você sabe + algo que você tem (ou é). TOTP, WebAuthn, hardware key.

RBAC / ABAC · Role / Attribute-Based Access Control

RBAC: permissão por role (admin/user). ABAC: permissão por atributos (location, device, time).

DPA · Data Processing Agreement

Contrato entre controlador e operador sobre como dados pessoais serão tratados (LGPD/GDPR).

DPIA / RIPD · Data Protection Impact Assessment / Relatório de Impacto

Avaliação documentada de riscos de tratamento de dados pessoais — obrigatória em alto risco.

DPO · Data Protection Officer / Encarregado

Pessoa designada como ponto de contato com ANPD e titulares (LGPD art. 41).

CSPM · Cloud Security Posture Management

Categoria de ferramenta que audita continuamente configuração de cloud (Wiz, Prowler, Lacework).

SCIM · System for Cross-domain Identity Management

Padrão para sincronizar usuários entre IdP (Okta) e apps (Slack, GitHub).

OIDC · OpenID Connect

Camada de identidade sobre OAuth 2.0. Federação moderna (GitHub Actions ↔ AWS).

WAF · Web Application Firewall

Filtro à frente da app que bloqueia padrões maliciosos (SQLi, XSS) por regras (Cloudflare, AWS WAF).

SIEM · Security Information & Event Management

Plataforma que centraliza logs e correlaciona eventos pra detectar comportamento anômalo (Datadog, Splunk, Elastic).

— ANÁLISE DE APLICAÇÃO

SAST · DAST · IAST · RASP

Quatro abordagens complementares para encontrar vulnerabilidades em aplicações. Não são concorrentes, são camadas.

SAST · STATIC APPLICATION SECURITY TESTING

O que faz: analisa o **código fonte** sem executá-lo. Encontra padrões perigosos: input não validado chegando em `eval()`, SQL com concatenação, hash MD5.

Quando rodar: em todo PR, no IDE, no CI.

Forças: roda cedo (shift-left), aponta linha exata, cobre 100% do código.

Limites: falsos positivos altos, não pega bug de runtime, não vê config externo.

Stack: Semgrep, CodeQL, SonarQube, Bandit, gosec, Brakeman.

DAST · DYNAMIC APPLICATION SECURITY TESTING

O que faz: ataca a **aplicação rodando** de fora — fuzzing, injeção, scan de portas, headers.

Quando rodar: em ambiente de staging, antes do deploy, scan agendado em prod.

Forças: simula atacante real, vê config + runtime + integrações.

Limites: só testa o que descobre crawling, lento, não vê código não exercitado.

Stack: OWASP ZAP, Burp Suite, Nuclei, sqlmap, Nikto.

IAST · INTERACTIVE APPLICATION SECURITY TESTING

O que faz: agente **dentro da aplicação** rastreia o fluxo de dados em runtime. Sabe se o input do usuário chegou no SQL com sanitização.

Quando rodar: durante QA, em ambiente integrado.

Forças: baixíssimo falso positivo, vê o caminho completo do dado.

Limites: só testa o que QA exercita, depende de cobertura de testes.

Stack: Contrast Security, Seeker (Synopsys), HCL AppScan IAST.

RASP · RUNTIME APPLICATION SELF-PROTECTION

O que faz: agente em **produção** que detecta e **bloqueia** ataques no momento que acontecem (não só detecta).

Quando rodar: em prod, sempre ligado.

Forças: defesa em tempo real, contexto completo, vê o atacante.

Limites: overhead de performance, requer agent em runtime.

Stack: Contrast Protect, Imperva RASP, Sqreen (descontinuado).

Quando usar cada um — pelo estágio

ESTÁGIO	SAST	DAST	IAST	RASP
IDE / dev local	●	—	—	—
Pull request	●	—	—	—
CI / build	●	—	—	—
QA / staging	●	●	●	—
Pré-deploy prod	—	●	—	—
Produção	—	●	—	●

Stack recomendada por porte

STARTUP / PEQUENA EQUIPE

Semgrep (SAST gratuito) em CI + OWASP ZAP (DAST) em staging mensal. WAF gerenciado (Cloudflare) para defesa em prod. Suficiente até PMF.

SCALE-UP / SÉRIE A+

Snyk Code + GitHub Advanced Security (SAST premium) + Burp Pro (DAST manual) + ZAP automatizado em CI. WAF gerenciado + bot management. Pentest anual.

ENTERPRISE / B2B SAAS REGULADO

Stack completa: Semgrep + CodeQL (SAST) + Burp Enterprise (DAST contínuo) + Contrast (IAST + RASP) + WAF + bot management. Time dedicado de AppSec. Bug bounty managed. Pentest trimestral.

O que não esperar

Nenhuma ferramenta dessa lista substitui:

- ☐ Threat modeling (humano, em quadro branco)
- ☐ Code review com lente de segurança
- ☐ Pentest profissional (criatividade humana)
- ☐ Cultura de "secure by default" no time

— TEMPLATES PRÁTICOS

Runbook & templates

Incident Response Runbook (resumido)

T+0 A T+1H · DETECÇÃO & TRIAGEM

- Confirmar incidente (não é falso positivo, falha de monitoração ou bug operacional)
- Acionar incident commander, abrir Slack channel #inc - AAAA-MM-DD
- Iniciar log de timeline (toda decisão registrada com timestamp)
- Classificar severidade preliminar (S1/S2/S3)

T+1 A T+4H · CONTENÇÃO

- Isolar sistemas afetados (kill session, rotacionar credentials, revogar tokens)
- Preservar evidências antes de destruir (snapshot de disco, dump de memória)
- Comunicar liderança (CEO/CTO) e jurídico interno

T+4 A T+24H · ERRADICAÇÃO & RECUPERAÇÃO

- Patch da vuln raiz; deploy em prod com revisão dupla
- Restaurar dados de backup limpo se necessário
- Monitoração reforçada por 7 dias

T+72H MÁXIMO · COMUNICAÇÃO ANPD & TITULARES

- Comunicação à ANPD em prazo razoável (LGPD art. 48 — geralmente interpretado como $\leq 72h$)
- Comunicação aos titulares afetados se houver risco relevante
- Considerar comunicação pública se vazamento foi massivo

T+7 A T+30 DIAS · POST-MORTEM

- Post-mortem blameless escrito (causa raiz, timeline, fatores contribuintes)
- Action items rastreáveis, com owner e prazo
- Revisão do próprio runbook (o que faltou, o que demorou)

DPIA simplificada (1 página)

Relatório de Impacto à Proteção de Dados – [Nome do Tratamento]

1. Descrição do tratamento

- Finalidade:
- Categorias de dados:
- Volume estimado:
- Categorias de titulares:

2. Base legal

- ☐ Consentimento ☐ Execução de contrato
- ☐ Obrigação legal ☐ Legítimo interesse
- ☐ Outra: _____

3. Avaliação de necessidade & proporcionalidade

- Por que precisamos coletar este dado?
- Existe alternativa menos invasiva?

4. Riscos identificados

- Risco 1: [descrição] – Probabilidade: B/M/A · Impacto: B/M/A
- Risco 2: ...

5. Medidas mitigadoras

- Técnicas: [criptografia, pseudonimização, controle de acesso]
- Organizacionais: [treinamento, política, contratos]

6. Decisão

- ☐ Aprovado ☐ Aprovado com ressalvas ☐ Rejeitado
- Responsável: _____ Data: _____ DPO: _____

Script de comunicação a titular pós-vazamento

Assunto: Comunicado importante sobre a sua conta na [Empresa]

Prezado(a) [Nome do titular],

Em [data], identificamos um incidente de segurança que pode ter afetado dados da sua conta. Levamos sua privacidade a sério e estamos te comunicando dentro do prazo previsto pela LGPD.

O que aconteceu: [descrição em linguagem clara]

Quais dados foram afetados: [lista específica – não generalize]

O que NÃO foi afetado: [lista – reduz pânico desnecessário]

O que estamos fazendo: [contenção, melhorias, parceiros]

O que recomendamos a você: [trocar senha, monitorar conta, etc]

Contato: Para qualquer dúvida, fale com nosso DPO em [email do DPO]. Você também pode acionar a ANPD se desejar.

Atenciosamente,

8 DIREITOS LGPD

Como implementar tecnicamente

LGPD art. 18 garante 8 direitos ao titular. Cada um exige implementação técnica concreta. Use isso como spec.

1. CONFIRMAÇÃO DE EXISTÊNCIA DE TRATAMENTO

- ☐ Endpoint GET /privacy/me retorna se existem dados do titular
- ☐ Resposta em até 15 dias (LGPD art. 19)
- ☐ Verificação de identidade antes de responder

2. ACESSO AOS DADOS

- ☐ Endpoint GET /privacy/data exporta tudo em formato legível
- ☐ Inclui dados de bancos de produção, analytics, logs, backups recuperáveis
- ☐ Job assíncrono se for grande volume; envio por link assinado de duração curta

3. CORREÇÃO DE DADOS INCOMPLETOS, INEXATOS OU DESATUALIZADOS

- ☐ Endpoint PATCH /privacy/data ou interface de auto-atendimento
- ☐ Audit log de toda correção (quem, quando, antes/depois)
- ☐ Propaga correção pra read replicas, caches e BI/data warehouse

4. ANONIMIZAÇÃO, BLOQUEIO OU ELIMINAÇÃO DE DADOS DESNECESSÁRIOS

- ☐ Endpoint POST /privacy/anonymize que aplica masking irreversível
- ☐ Estratégia clara: anonimização vs pseudonimização vs deleção
- ☐ Mantém dados mínimos para obrigação legal (fiscal, antifraude)

5. PORTABILIDADE DOS DADOS

- ☐ Endpoint GET `/privacy/export` retorna formato estruturado (JSON, CSV)
- ☐ Inclui apenas dados fornecidos pelo titular ou gerados por sua atividade
- ☐ Documentação do schema pública pra concorrente conseguir importar

6. ELIMINAÇÃO DOS DADOS TRATADOS COM CONSENTIMENTO

- ☐ Endpoint DELETE `/privacy/account` dispara workflow de deleção
- ☐ Soft delete imediato + hard delete após período de carência
- ☐ Propaga deleção pra todos os sistemas: CRM, e-mail marketing, analytics, BI
- ☐ Notifica sub-operadores via API ou webhook configurado

7. INFORMAÇÃO SOBRE COMPARTILHAMENTO

- ☐ Política de Privacidade lista todos os sub-operadores (Amplitude, Stripe, Sendgrid, etc.)
- ☐ Endpoint GET `/privacy/sharing` retorna lista atualizada por API
- ☐ Notificação quando sub-operadores mudam (versionamento da política)

8. REVOGAÇÃO DO CONSENTIMENTO

- ☐ Interface de granular consent management: marketing, analytics, terceiros
- ☐ Revogação com mesma facilidade que aceitação (1 clique)
- ☐ Estado de consentimento auditável com timestamp e IP
- ☐ Propagação imediata: deixa de enviar e-mail no próximo ciclo

Stack recomendada para LGPD operacional

- **OneTrust / DataPrivacy / Privally** — gestão de cookies, consent, mapeamento de tratamento
- **Vanta / Drata** — automação SOC 2 + LGPD (caro mas vale enterprise)
- **Bytebase** — data masking automático em ambientes não-prod
- **Open-source:** Klaro (consent), build interno do `/privacy/*` endpoints

Erros comuns

- ☐ "Estou com seu pedido em análise" sem prazo claro — viola art. 19
- ☐ Soft delete que sobrevive eternamente nos backups — viola art. 16
- ☐ Exigir CPF e RG pra exercer direito (excesso de coleta pra exercer direito)
- ☐ Não propagar deleção pra Mailchimp / HubSpot / analytics
- ☐ Cobrar pela exportação dos dados — direito é gratuito